```
#define BLYNK_TEMPLATE_ID "TMPL65dgZJ0Wu"
#define BLYNK_TEMPLATE_NAME "SDMS"
#define BLYNK_AUTH_TOKEN "RmLdaYTx3z17WE9GGXOLg25HlPpOOzU3"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <ESP32Servo.h>
#include <Preferences.h>

char ssid[] = "";
char pass[] = "";

// Pin definitions
#define SERVO_PIN 14
#define REED_PIN 26
#define RED_LED 4
#define GREEN_LED 5
#define BUZZER_PIN 23
#define BUTTON_PIN 12

Servo lockServo;
BlynkTimer timer;
Preferences prefs;

// System variables
bool isDoorLocked = false;
bool lastReedState = false;
bool silentMode = false;
bool systemActive = true;
bool alarmActive = false;
int servoLockAngle = 0;
int servoUnlockAngle = 180;

unsigned long buttonPressTime = 0;
bool buttonHeld = false;

// ---- Persistent storage helpers ----
// Save the lock state to ESP32's internal NVS (flash) memory
// so it survives power loss.
void saveLockState(bool locked) {
prefs.putBool("doorLocked", locked);
}

// Sync slider values when ESP32 connects to Blynk
BLYNK_CONNECTED() {
Blynk.syncVirtual(V3);
Blynk.syncVirtual(V4);
Blynk.syncVirtual(V5);
// Report the real lock state to the app once connected
Blynk.virtualWrite(V0, isDoorLocked ? 1 : 0);
```

```
}

// Servo move function
void moveServo(int angle) {
lockServo.attach(SERVO_PIN, 400, 2600);
lockServo.write(angle);
delay(800);
lockServo.detach();
}

// System buzzer - only for system events
void systemBuzzer(int times) {
for (int i = 0; i < times; i++) {
digitalWrite(BUZZER_PIN, HIGH);
delay(300);
digitalWrite(BUZZER_PIN, LOW);
delay(200);
}
}

// Start alarm
void startAlarm() {
alarmActive = true;
Serial.println("ALARM STARTED!");
}

// Stop alarm
void stopAlarm() {
alarmActive = false;
digitalWrite(BUZZER_PIN, LOW);
Serial.println("Alarm stopped");
}

// Run alarm in loop - non-blocking
void runAlarm() {
if (!alarmActive) return;
if (silentMode) return;
static unsigned long lastBeep = 0;
static bool buzzerState = false;
unsigned long now = millis();
if (now - lastBeep >= 100) {
lastBeep = now;
buzzerState = !buzzerState;
digitalWrite(BUZZER_PIN, buzzerState ? HIGH : LOW);
}
}

// Lock the door
void lockDoor() {
if (!systemActive) return;
```

```
moveServo(servoLockAngle);
isDoorLocked = true;
digitalWrite(RED_LED, HIGH);
digitalWrite(GREEN_LED, LOW);
Blynk.virtualWrite(V0, 1);
saveLockState(true);
Serial.print("Door LOCKED at angle: ");
Serial.println(servoLockAngle);
}

// Unlock the door
void unlockDoor() {
if (!systemActive) return;
moveServo(servoUnlockAngle);
isDoorLocked = false;
digitalWrite(RED_LED, LOW);
digitalWrite(GREEN_LED, HIGH);
Blynk.virtualWrite(V0, 0);
saveLockState(false);
Serial.print("Door UNLOCKED at angle: ");
Serial.println(servoUnlockAngle);
}

// Used at startup: before Blynk connects, restores the door/LEDs to the
// last saved state. Unlike lockDoor()/unlockDoor(), it doesn't call
// Blynk.virtualWrite (connection may not be ready yet) and ignores
// systemActive - it just physically matches the saved state.
void restoreDoorState(bool locked, int angle) {
moveServo(angle);
isDoorLocked = locked;
digitalWrite(RED_LED, locked ? HIGH : LOW);
digitalWrite(GREEN_LED, locked ? LOW : HIGH);
Serial.print("Door state restored: ");
Serial.println(locked ? "LOCKED" : "UNLOCKED");
}

// Blynk: Lock control command
BLYNK_WRITE(V0) {
if (!systemActive) return;
int val = param.asInt();
if (val == 1) lockDoor();
else unlockDoor();
}

// Blynk: Buzzer/Alarm control
// V2 = 1 -> start alarm
// V2 = 0 -> stop alarm
BLYNK_WRITE(V2) {
if (!systemActive) return;
if (param.asInt() == 1) {
```

```
startAlarm();
} else {
stopAlarm();
}
}

// Blynk: Set lock angle
BLYNK_WRITE(V3) {
servoLockAngle = param.asInt();
Serial.print("Lock angle set to: ");
Serial.println(servoLockAngle);
if (isDoorLocked) {
moveServo(servoLockAngle);
}
}

// Blynk: Set unlock angle
BLYNK_WRITE(V4) {
servoUnlockAngle = param.asInt();
Serial.print("Unlock angle set to: ");
Serial.println(servoUnlockAngle);
if (!isDoorLocked) {
moveServo(servoUnlockAngle);
}
}

// Blynk: Silent mode control
BLYNK_WRITE(V5) {
silentMode = param.asInt();
if (silentMode) stopAlarm();
Serial.println(silentMode ? "Silent mode ON" : "Silent mode OFF");
}

// Reed sensor check
void checkDoorSensor() {
if (!systemActive) return;
bool reedState = digitalRead(REED_PIN);

// Only when state changes
if (reedState != lastReedState) {
Blynk.virtualWrite(V1, reedState ? 1 : 0);

// Forced entry detection
if (isDoorLocked && reedState == HIGH) {
Serial.println("WARNING: Forced entry detected!");
Blynk.logEvent("security_alert", "WARNING: Door forced open!");
startAlarm();
}

// Normal door open
```

```
if (!isDoorLocked && reedState == HIGH) {
Blynk.logEvent("door_opened", "Door opened");
Serial.println("Door opened");
}

// Door closed
if (reedState == LOW) {
Blynk.logEvent("door_closed", "Door closed");
Serial.println("Door closed");
}

lastReedState = reedState;
}
}

// Physical button check
void checkButton() {
if (digitalRead(BUTTON_PIN) == LOW) {
if (buttonPressTime == 0) {
buttonPressTime = millis();
delay(50);
}

// 3 seconds hold = system on/off
if (millis() - buttonPressTime > 3000 && !buttonHeld) {
buttonHeld = true;
systemActive = !systemActive;

if (systemActive) {
Serial.println("System ON");
digitalWrite(RED_LED, HIGH);
delay(200);
digitalWrite(RED_LED, LOW);
delay(200);
digitalWrite(GREEN_LED, HIGH);
delay(200);
digitalWrite(GREEN_LED, LOW);
systemBuzzer(2);
// Return to the last known lock state instead of forcing unlock
if (isDoorLocked) lockDoor(); else unlockDoor();
} else {
Serial.println("System OFF");
systemBuzzer(3);
stopAlarm();
digitalWrite(RED_LED, LOW);
digitalWrite(GREEN_LED, LOW);
lockServo.detach();
}
}
```

```
} else {
// Short press = lock/unlock
if (buttonPressTime > 0 && !buttonHeld) {
if (systemActive) {
if (isDoorLocked) unlockDoor();
else lockDoor();
}
}
buttonPressTime = 0;
buttonHeld = false;
}
}

void setup() {
Serial.begin(115200);

pinMode(RED_LED, OUTPUT);
pinMode(GREEN_LED, OUTPUT);
pinMode(BUZZER_PIN, OUTPUT);
pinMode(BUTTON_PIN, INPUT_PULLUP);
pinMode(REED_PIN, INPUT_PULLUP);

// Read the saved lock state from flash memory
prefs.begin("sdms", false);
bool savedLocked = prefs.getBool("doorLocked", false); // default on first boot: unlocked

Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

timer.setInterval(2000L, checkDoorSensor);

// Instead of always forcing unlockDoor(), restore the state
// the door was in before power was lost.
restoreDoorState(savedLocked, savedLocked ? servoLockAngle : servoUnlockAngle);

Serial.println("SDMS System Ready!");
}

void loop() {
Blynk.run();
timer.run();
checkButton();
runAlarm();
}
```